



Zoho Schools for Graduate Studies



Notes

FLAVOURS AND SUB-FLAVOURS OF COLLECTION

CORE KEY INTERFACES OF THE COLLECTIONS FRAMEWORK :

1. Collection
2. List
3. Queue
4. Set
5. Map
6. Navigable Set
7. Sorted Set
8. Sorted Map
9. Navigable Map

Note : Collection is the root Interface in the collection hierarchy

BASIC FLAVOURS OF COLLECTION :

Collections come in four basic flavours :

1. Lists - Classes that implements List
2. Sets - Classes that implements Set
3. Maps - Classes that implements Map
4. Queue - Classes that implements Queue

THE CORE CONCRETE CLASSES OF THE COLLECTIONS FRAMEWORK :

Lists	Sets	Queues	Maps	Utilities
ArrayList	HashSet	Priority Queue	HashMap	Collections
Vector	LinkedHashSet	ArrayDeque	HashTable	Arrays
LinkedList	TreeSet		TreeMap	
Stack			LinkedHash Map	

ORDERED vs SORTED :

- An **ordered collection** means that the elements of the collection have a specific order. The Order is independent of the value. A **List** is an example.
- A **sorted collection** means that not only does the collection have order, but the order depends on the value of the element. A **SortedSet** is an example.
- In Contrast, a **collection without any order** can maintain the elements in any order. A **Set** is an example.

LIST IMPLEMENTATION CLASSES :

List	Ordered	Ordered by	Sorted
ArrayList	Yes	Index	No
Vector	Yes	Index	No
LinkedList	Yes	Index	No

SET IMPLEMENTATION CLASSES :

Set	Ordered	Ordered by	Sorted
HashSet	No		No
LinkedHashSet	Yes	Insertion Order	No
TreeSet	Yes	Sorted	By natural order or custom order

MAP IMPLEMENTATION CLASSES :

Map	Ordered	Ordered by	Sorted
HashMap	No		No
HashTable	No		No
TreeMap	Sorted		By Natural Order or Custom Order
LinkedHashMap	Yes	Insertion Order	No

QUEUE IMPLEMENTATION CLASSES :

Queue	Ordered	Ordered by	Sorted
Priority Queue	Sorted		By Natural Order or Custom Order
Array Deque	Yes	Position	No

ARRAYS VS COLLECTIONS :

Key	Arrays	Collection
Size	Arrays or Fixed in size i.e. static	Dynamic in size
Memory Usage	Inefficient use of memory	Efficient use of memory
Data Type	Stores only homogeneous data	Can store both homogeneous and heterogeneous data
Primitive Storage	Can store both objects and primitives	Can store only object types
Performance	Better in Performance	Relatively low in Performance